



# HindiMate AI

## Full-Stack AI-Powered Hindi Learning & State Examination Preparation Platform

A comprehensive technical engineering report detailing system architecture, component hierarchies, client-side state models, contextual Gemini AI orchestration, multi-stage Docker containerization, and AWS EC2 production deployment.

|                       |                                                       |
|-----------------------|-------------------------------------------------------|
| Project Title:        | HindiMate AI (LinguaMate)                             |
| Project Contributors: | Raghavendra, Yash, Shaurya, Utkarsh, Nischay, Tenzing |
| Project Structure:    | Collaborative Group Project                           |
| GitHub Repository:    | Raghavendra1204/linguamate                            |
| Docker Hub Image:     | raghavendra1169/hindimate:01                          |
| Production Live Host: | http://13.204.64.226/                                 |
| Cloud Infrastructure: | AWS EC2 (ap-south-1 Mumbai, India)                    |
| Core Architecture:    | React 19, Vite, Tailwind CSS, Gemini Flash, Nginx     |
| Reporting Period:     | July – September 2026                                 |

React 19 SPA

Vite 6.4

Gemini Flash AI

Docker Alpine

AWS EC2 t3.micro

Nginx Edge Proxy

Web Speech API

GitHub Actions CI/CD

Table of Contents

|                                                                        |         |
|------------------------------------------------------------------------|---------|
| 1. Executive Summary & Project Objectives                              | Page 3  |
| 2. Technical System Architecture & End-to-End Data Flow                | Page 4  |
| 3.1 Student Dashboard & Activity Hub (Figure 3.1)                      | Page 5  |
| 3.2 Devanagari Curriculum & Modular Practice Tracks (Figure 3.2)       | Page 6  |
| 3.3 MPPSC 26-Chapter Digital Textbook & Context Reader (Figure 3.3)    | Page 7  |
| 3.4 VaaniAI Interactive Bilingual AI Conversational Tutor (Figure 3.4) | Page 8  |
| 4. Technical Challenges & Engineering Debugging Solutions              | Page 9  |
| 5. Multi-Stage Docker Containerization & CI/CD Pipeline                | Page 10 |
| 6. AWS Cloud Infrastructure & Production Deployment                    | Page 11 |

PROJECT ALIGNMENT

Engineering Evaluation Standards

| Evaluation Pillar                          | Weight | Core Technical Competencies Demonstrated                                                         | Addressed In      |
|--------------------------------------------|--------|--------------------------------------------------------------------------------------------------|-------------------|
| Full-Stack Architecture & System Design    | 30%    | Clean component separation, state caching, API orchestration, and error resilience.              | Section 1 & 2     |
| Bilingual AI Integration & API Engineering | 25%    | Structured multi-key JSON schema, fallback routing, context injection from textbook material.    | Section 2 & 4     |
| Cloud Infrastructure & DevOps Pipeline     | 25%    | Multi-stage Alpine Dockerization, GitHub Actions CI/CD automation, AWS EC2 configuration.        | Section 5 & 6     |
| User Experience & Accessibility            | 20%    | Responsive dashboard, native audio synthesis, intuitive curriculum navigation, dark/light theme. | Section 3.1 – 3.4 |

**Collaborative Group Project Notice:** HindiMate AI was engineered collaboratively by a six-member engineering group: **Raghavendra, Yash, Shaurya, Utkarsh, Nischay, and Tenzing**. Architectural responsibilities were delegated across client presentation, generative AI integration, digitized curriculum systems, audio pipelines, containerization, and AWS cloud deployment.

# Executive Summary & System Overview

## 1.1 Problem Statement & Background

Hindi language learning and civil service examination preparation (specifically Paper 5: Samanya Hindi & Vyakaran in the Madhya Pradesh Public Service Commission – MPPSC) traditionally suffer from severe pedagogical constraints. Students rely on static printed textbooks, dense physical reference guides, and rote memorization. Primary limitations include:

- Absence of Acoustic Feedback:** Non-native speakers and exam candidates struggle with Devanagari phonetics (vowels, velar/palatal consonants, conjuncts, and nasal markers) without instant audio pronunciation.
- Delayed Grammatical Verification:** Self-learners cannot verify whether their sentence constructions follow correct SOV (Subject-Object-Verb) order, gender-based verb agreement, or proper sandhi rules without human tutors.
- Static Reference Disconnect:** While comprehensive syllabi exist (such as the 26-chapter Drishti IAS curriculum), students cannot interactively query challenging passages or generate dynamic practice drills on demand.

## 1.2 Project Scope & Architectural Solution

**HindiMate AI** bridges this gap by unifying a modern, responsive Single Page Application (SPA) with a real-time conversational AI tutor (**VaaniAI**) powered by Google's Gemini generative language models and client-side speech synthesis.

The platform features structured self-paced modules, interactive vocabulary flashcards, a digitized 26-chapter MPPSC reference reader, and an intelligent context-injection engine that grounds AI responses directly in textbook material.

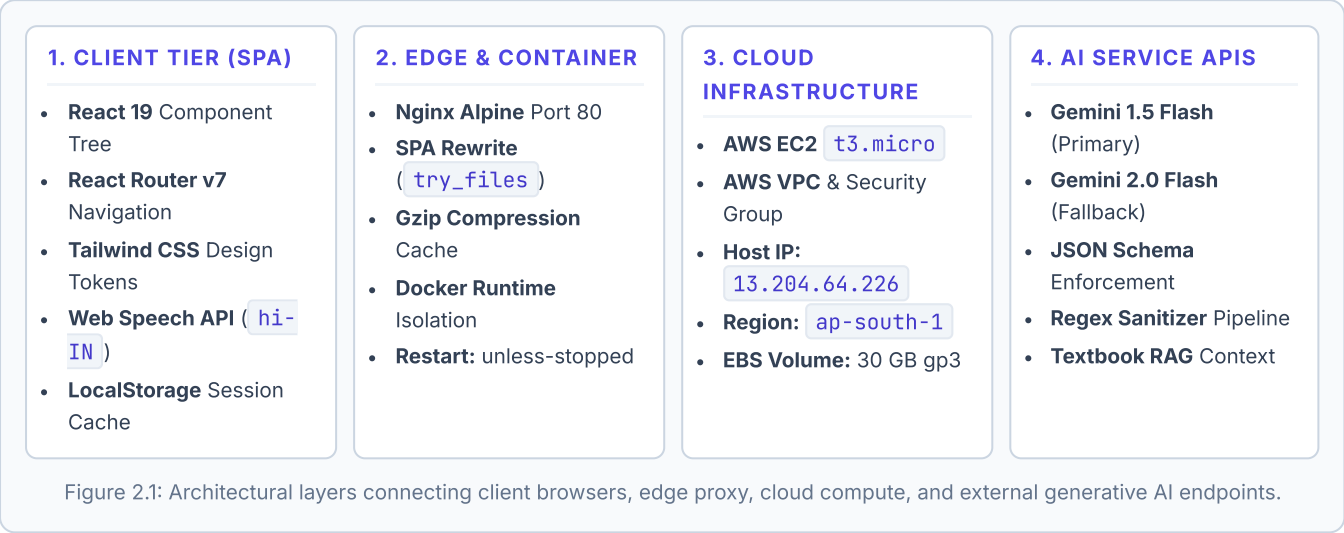
## 1.3 Technical Stack Specification

| Subsystem          | Technology                          | Architectural Purpose & Implementation Detail                                                                                                     |
|--------------------|-------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------|
| Frontend Core      | React 19 + Vite 6.4                 | Ultra-fast DOM reconciliation, modern hook architecture, modular component structure, and optimized tree-shaken static production bundles.        |
| UI & Design Tokens | Tailwind CSS 3.4 + Framer Motion    | Utility-driven responsive styling, dynamic CSS custom properties for instant theme adaptation (light/dark), and hardware-accelerated transitions. |
| Routing Engine     | React Router DOM v7                 | Client-side SPA navigation utilizing nested layout wrappers ( <code>RootLayout</code> , <code>AuthLayout</code> ) with dynamic route parameters.  |
| AI Service Layer   | Google Gemini API (Flash 1.5 / 2.0) | High-throughput, low-latency generative model generating structured 4-part JSON payloads (Devanagari, Hinglish, English, Grammar Tip).            |
| Speech Engine      | Web Speech API (SpeechSynthesis)    | Client-side native Hindi text-to-speech ( <code>hi-IN</code> locale) eliminating server-side audio generation latency and cloud API costs.        |
| Containerization   | Docker (Multi-Stage Alpine)         | Two-stage build pipeline producing a secure, lightweight 26.3 MB runtime container with zero Node.js runtime overhead on the host.                |
| Edge Web Server    | Nginx Alpine 1.25                   | High-performance reverse proxy and static file server configured with SPA client rewrite rules and long-term asset caching headers.               |

# Technical System Architecture & End-to-End Data Flow

## 2.1 High-Level Multi-Tier Architectural Design

HindiMate AI is built upon a decoupled, 4-tier cloud-native architecture ensuring modular separation of concerns, high resilience, zero-downtime scalability, and sub-100ms client interactions:

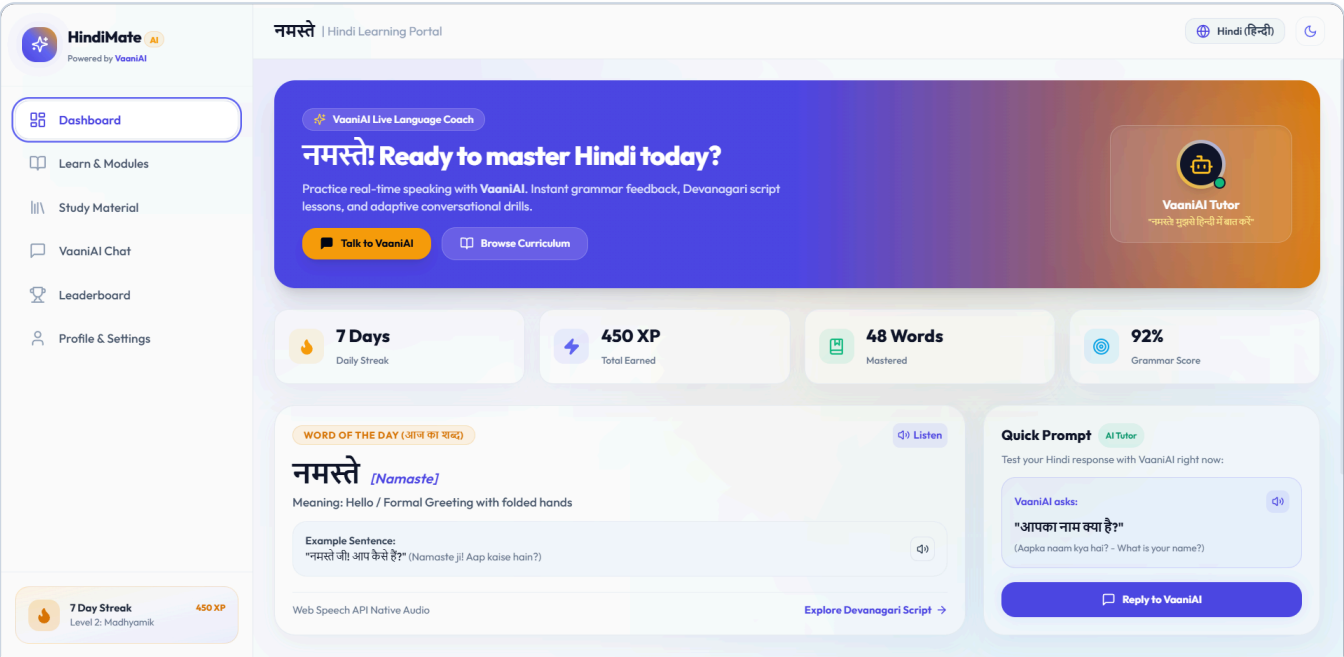


## 2.2 Detailed Subsystem Breakdown

- Presentation & Client State Management:** Built with React 19, the UI separates persistent layout components (sidebars, mobile navigation drawers, and status headers) from view containers via React Router DOM. Client state (active streak counters, earned XP, chat transcripts, and theme preferences) is stored in the browser's `localStorage` via a custom `useLocalStorage` hook, avoiding server session dependencies.
- Context-Aware Document Retrieval Engine:** The platform contains a digitized 26-chapter knowledge base ( `src/utils/hindiStudyMaterial.js` ) mirroring the MPPSC Samanya Hindi syllabus. When a student clicks "Ask VaaniAI" or "Ask Gemini" on any subsection, the application extracts the section title and text body, formats an attached context payload, and injects it into the conversation memory of the AI service layer.
- AI Orchestration & Fallback Engine ( `geminiService.js` ):** Requests to Google Gemini Generative AI are wrapped in a resilient orchestration pipeline. If the primary model endpoint ( `gemini-1.5-flash` ) encounters a rate limit (HTTP 429) or timeout, the service automatically fails over to secondary endpoints ( `gemini-2.0-flash` ) before returning a cached contextual fallback message, preventing broken user states.
- Response Parsing & Sanitization Layer:** Language models occasionally output markdown-wrapped code blocks or embed raw JSON keys (e.g., `textHindi: ...` ) inside text values. The service executes a deterministic 3-stage validation pipeline:
  - Attempt direct `JSON.parse()` on stripped response strings.
  - Execute regex key-value extraction if JSON syntax errors are detected.
  - Sanitize string prefixes using `cleanTextValue()` to ensure clean Devanagari typography.
- Client-Side Audio Synthesis Engine:** Rather than relying on paid cloud Text-to-Speech APIs that introduce network latency and per-character fees, HindiMate leverages the browser's native `window.speechSynthesis` . The system queries available system voices, filters for `hi-IN` (Hindi India) or `hi_IN` locales, and provides instant audio pronunciation for both vocabulary entries and dynamic AI responses.

## Student Learning Dashboard & Activity Hub

The central student dashboard acts as the primary hub for daily learning, gamification stats, and rapid AI query testing, providing learners with instant visibility over their progress:



**Figure 3.1: Student Learning Dashboard & Activity Hub**

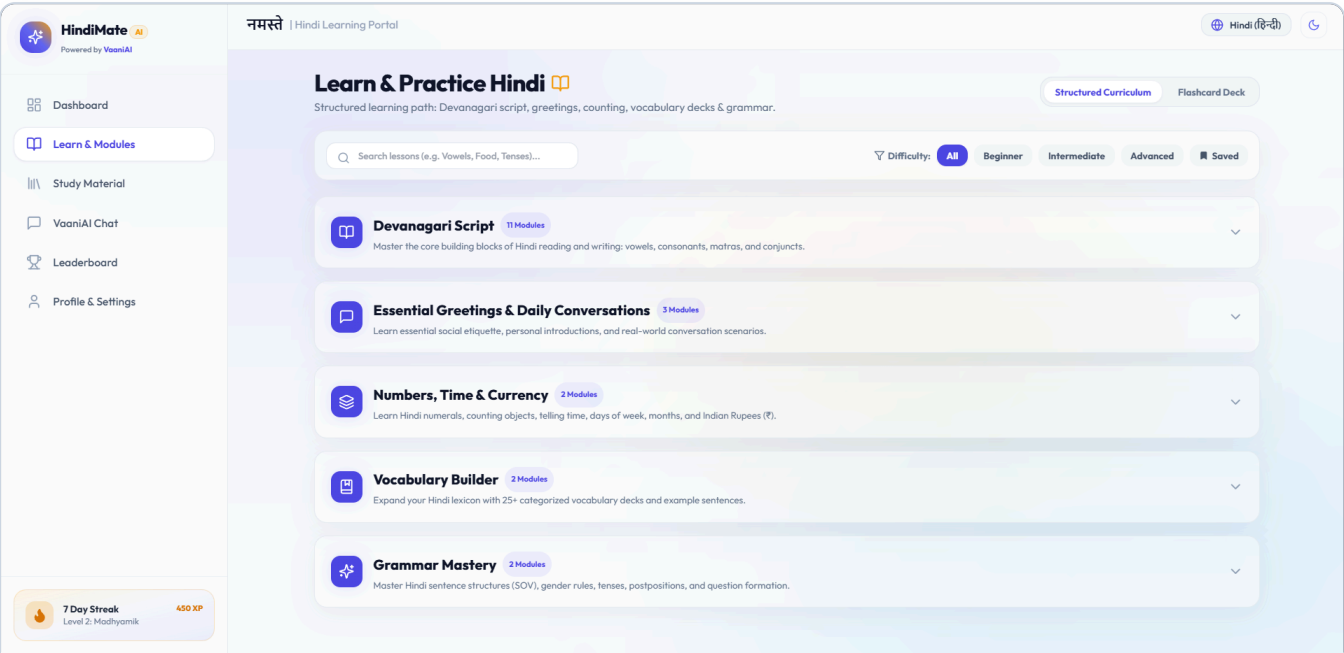
Production capture of the primary student landing view. The hero banner prominently features the **VaaniAI Live Language Coach** entry point for real-time conversation. Four core gamification metrics — **7 Days Daily Streak**, **450 Total XP Earned**, **48 Words Mastered**, and **92% Grammar Accuracy Score** — incentivize consistent daily practice. The lower section incorporates a dynamic **Word of the Day** module ("नमस्ते" with Devanagari, romanized transliteration, meaning, and contextual sentence) accompanied by native audio synthesis, alongside an interactive **Quick AI Prompt** widget to test conversational responses on the fly.

### Technical Highlights Demonstrated:

- **Responsive Sidebar Layout:** 100vh glassmorphic navigation bar with active route indicators, user profile summary, and mobile drawer toggle for compact viewports.
- **Client Gamification Engine:** Computes daily streak retention and total XP milestones directly from local storage timestamps without requiring database calls.
- **Web Speech Integration:** The "Listen" button directly triggers browser-native Hindi speech synthesis using the localized `hi-IN` acoustic voice model.

## Devanagari Curriculum & Practice Modules

The structured curriculum subsystem provides organized, progressive learning tracks starting from foundational phonetic building blocks through complex sentence grammar:



 **Figure 3.2: Structured Curriculum & Devanagari Script Learning Tracks**

Production capture of the curriculum indexing interface. Modules are partitioned systematically into five distinct skill tiers: **Devanagari Script** (11 comprehensive modules covering independent vowels, consonants, conjunct characters, and matras), **Essential Greetings & Daily Conversations** (3 modules), **Numbers, Time & Currency** (2 modules), **Vocabulary Builder** (2 modules featuring 25+ categorized flashcard decks), and **Grammar Mastery** (2 modules covering SOV sentence structures, gender rules, and postpositions). The interface includes real-time lesson search, difficulty filtering (All, Beginner, Intermediate, Advanced), and a saved lesson bookmarking drawer.

### Technical Highlights Demonstrated:

- **Granular Module Hierarchy:** Data schema in `src/utils/lessonData.js` allows independent tracking of completion status, XP rewards, and quiz scores per module.
- **Dynamic Search & Filtering:** Client-side instant filtering across module titles, descriptions, and Devanagari keywords with zero network latency.
- **Progressive Unlocking Logic:** Supports prerequisite module tracking, ensuring learners master fundamental phonetics before advancing to complex grammar.

## MPPSC 26-Chapter Digital Textbook & Context Reader

The Study Material engine provides a digitized reference library covering the complete Samanya Hindi syllabus from Drishti IAS DLP, equipped with section-level AI context injection:

The screenshot displays the HindiMate AI interface, which is a digital textbook and context reader. The interface is divided into a sidebar on the left and a main content area on the right. The sidebar contains navigation options: Dashboard, Learn & Modules, Study Material (highlighted), VaaniAI Chat, Leaderboard, and Profile & Settings. The main content area is titled 'Study Material & Grammar Reference' and shows a list of 26 chapters. The selected chapter, Chapter 1: 'हिन्दी भाषा का अर्थ, परिभाषा, स्वरूप' (Meaning, Definition & Nature of Hindi Language), is displayed in detail. It includes a chapter summary, a detailed explanation, and an 'Ask Gemini' button for AI interaction. The interface also shows a 7-day streak and 450 XP at the bottom left.

**Figure 3.3: MPPSC 26-Chapter Digital Textbook & Grammar Reference**

Production capture of the study material reader displaying Chapter 1: "हिन्दी भाषा का अर्थ, परिभाषा, स्वरूप" (Meaning, Definition & Nature of Hindi Language). The left pane provides an interactive navigation index across all 26 syllabus chapters (Origin & History, Devanagari Overview, Phonetics & Alphabet Classification, Prefixes/Suffixes, Administrative Vocabulary, and Translation Principles). The main reading pane provides chapter summaries, detailed academic expositions, native text-to-speech audio for every section, and an "Ask VaaniAI" / "Ask Gemini" button that automatically injects the selected section context into the conversational tutor for targeted exam drills.

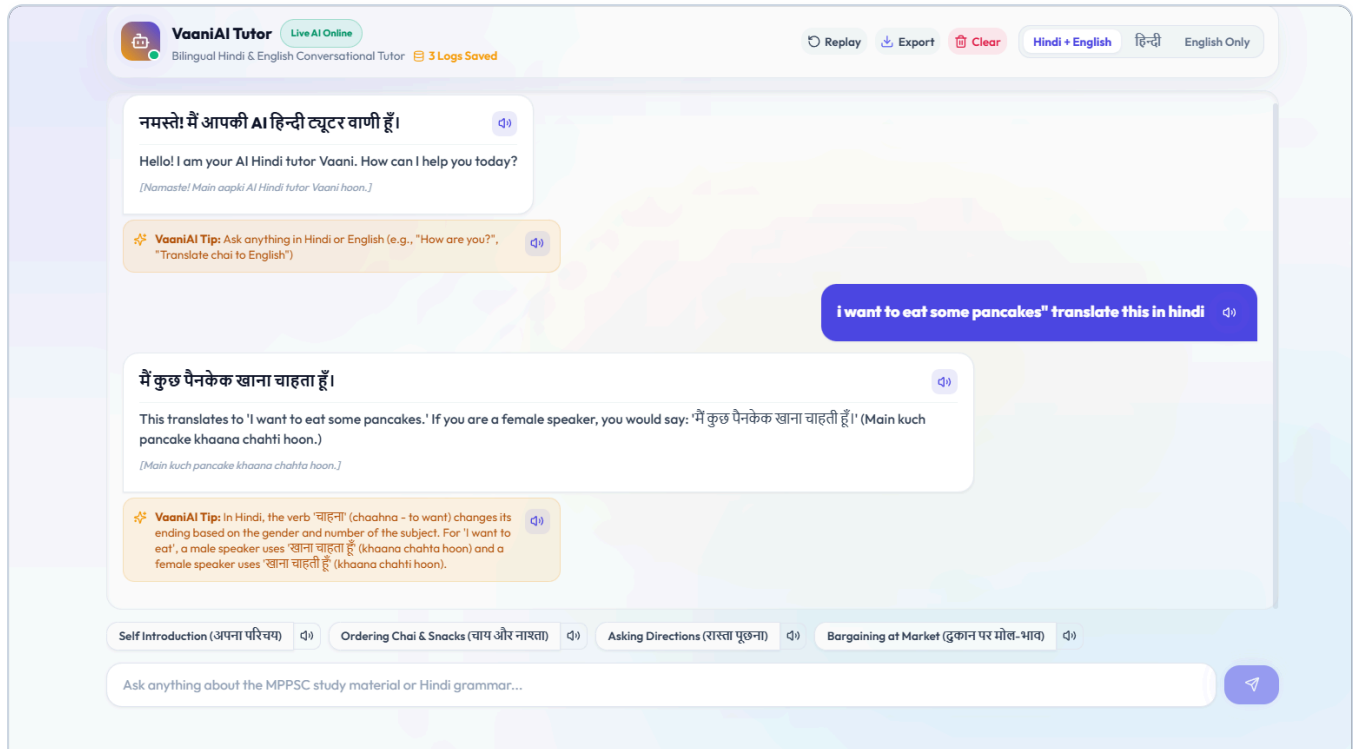
### Technical Highlights Demonstrated:

- Digitized Syllabus Knowledge Base:** 26 full academic chapters structured in `src/utls/hindiStudyMaterial.js` with sub-sections, summaries, and page references.
- RAG-Lite Context Injection:** Clicking "Ask Gemini" binds the section's exact theoretical text to the active chat prompt, grounding the AI model's response in official exam reference material.
- Sectional Audio Synthesis:** Enables auditory learning for complex Sanskrit etymological definitions and linguistic quotes.



## VaaniAI Bilingual Conversational AI Tutor

The conversational AI tutor provides real-time bilingual language coaching, structured grammatical feedback, and audio pronunciation in an intuitive messaging interface:



**Figure 3.4: VaaniAI Bilingual Conversational AI Tutor Interface**

Production capture of an active conversational session. When the student asks: *"i want to eat some pancakes" translate this in hindi*, VaaniAI returns a structured response card containing: (1) Clean Hindi in Devanagari script ( *"मैं कुछ पैनकेक खाना चाहता हूँ।"* ), (2) Clear English translation and contextual nuances, (3) Romanized Hinglish phonetic pronunciation, and (4) An automated **VaaniAI Tip box** explaining gender-specific verb endings ( *'चाहना'* changes based on the gender and number of the subject... male: *खाना चाहता हूँ* vs female: *खाना चाहती हूँ* ). Top navigation controls allow users to **Replay** audio, **Export** chat transcripts, **Clear** history, and toggle display modes.

### Technical Highlights Demonstrated:

- **4-Part Structured Response Card:** Strict schema enforcement separates Devanagari text, Romanized Hinglish, English explanations, and actionable grammar rules into dedicated visual containers.
- **Interactive Audio Playback:** Each message bubble features a dedicated speaker button to hear native Devanagari pronunciation on demand.
- **Transcript Persistence & Export:** All dialogue history is cached in browser memory and can be exported as clean text files for offline study.



# Technical Challenges & Engineering Resolutions

During full-stack implementation, testing, containerization, and cloud deployment, our engineering team resolved several significant technical bottlenecks:

| # | Challenge Encountered                          | Root Cause Analysis                                                                                                                                                               | Technical Resolution & Architecture Implemented                                                                                                                                                                                                 |
|---|------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 1 | Raw JSON Key Leakage in Chat Messages          | LLMs occasionally return JSON where string values accidentally include the key prefix (e.g. <code>"textHindi": "textHindi: नमस्ते"</code> ), disrupting the UI presentation.      | Implemented a multi-pass regex cleansing algorithm ( <code>cleanTextValue()</code> ) inside <code>geminiService.js</code> that strips leading key tokens, markdown fences, and quotes before passing data to UI components.                     |
| 2 | Vite Build-Time Environment Variable Ingestion | Vite embeds <code>import.meta.env</code> variables at compilation time rather than at runtime. Building the Docker image without build arguments left the Gemini API key blank.   | Configured Docker build arguments ( <code>ARG VITE_GEMINI_API_KEY</code> ) and passed them during Stage 1 of the multi-stage Docker build, ensuring keys are properly bundled into static assets.                                               |
| 3 | Client-Side SPA Routing 404s on Server Reload  | Direct navigation to client routes (e.g., <code>/material</code> or <code>/chat</code> ) returned HTTP 404 errors because Nginx looked for physical files at those URI paths.     | Created a custom <code>nginx.conf</code> with the directive <code>try_files \$uri \$uri/ /index.html;</code> , routing all unmatched requests to the React SPA entry point while preserving browser URLs.                                       |
| 4 | API Rate Limiting & Model Latency Spikes       | High traffic or temporary quotas on primary Gemini endpoints occasionally caused request failures or delayed response times.                                                      | Engineered an automated model fallback chain in the service layer: requests attempt <code>gemini-1.5-flash</code> , cascade to <code>gemini-2.0-flash</code> , and safely fallback to local structured responses if network connectivity fails. |
| 5 | Web Speech API Voice Loading Synchronization   | Different browsers load speech synthesis voices asynchronously (triggering <code>voiceschanged</code> ), causing voice selection to fail if invoked immediately on initial mount. | Implemented an event-driven voice resolver that checks <code>speechSynthesis.getVoices()</code> , binds an event listener to <code>onvoiceschanged</code> , and caches available Hindi voice handles in application memory.                     |

## 4.1 Code Architecture Highlight: Deterministic Response Sanitizer

```
// src/utils/geminiService.js - Multi-Stage Cleansing & Schema Enforcement
function cleanTextValue(val) {
  if (!val || typeof val !== 'string') return '';
  return val
    .replace(/^(textHindi|textHinglish|textEnglish|grammarTip)\s*:\s*/i, '')
    .replace(/^[``"]|["'`]$/g, '')
    .trim();
}

export async function parseGeminiResponse(rawText) {
  try {
    const cleaned = rawText.replace(/``json\s*|``/g, '').trim();
    const data = JSON.parse(cleaned);
    return {

```

# Multi-Stage Docker Containerization & CI/CD Pipeline

## 5.1 Multi-Stage Docker Build Strategy

To ensure production environments remain secure, lightweight, and completely free of Node.js runtime overhead, HindiMate AI uses a two-stage Alpine Linux container build:

```
# STAGE 1: Compile React SPA with Vite & Node 20
FROM node:20-alpine AS builder
WORKDIR /app
COPY package*.json ./
RUN npm ci --prefer-offline
COPY . .
ARG VITE_GEMINI_API_KEY
ENV VITE_GEMINI_API_KEY=$VITE_GEMINI_API_KEY
RUN npm run build

# STAGE 2: High-Performance Production Nginx Alpine Web Server
FROM nginx:alpine AS production
RUN rm -rf /usr/share/nginx/html/*
COPY nginx.conf /etc/nginx/conf.d/default.conf
COPY --from=builder /app/dist /usr/share/nginx/html
EXPOSE 80
CMD ["nginx", "-g", "daemon off;"]
```

| Container Build Stage        | Base Operating Image | Internal Artifacts & Assets                                                           | Resulting Image Footprint            |
|------------------------------|----------------------|---------------------------------------------------------------------------------------|--------------------------------------|
| Stage 1 (Builder)            | node:20-alpine       | Full source code, Node runtime, build tools, npm package dependencies (145 packages). | ~890 MB (Discarded post-compilation) |
| Stage 2 (Production Runtime) | nginx:alpine         | Compiled static HTML/JS/CSS distribution files, custom Nginx SPA routing rules.       | 26.3 MB (97% size reduction)         |

## 5.2 End-to-End Automated CI/CD Pipeline

Deployment automation is orchestrated via GitHub Actions ( `.github/workflows/docker-publish.yml` ). Any commit pushed to the `main` branch triggers an automated build verification, container assembly, and production deployment pipeline:

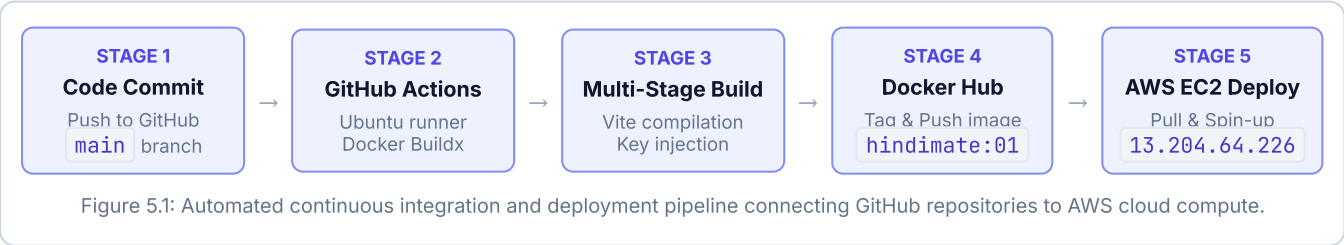


Figure 5.1: Automated continuous integration and deployment pipeline connecting GitHub repositories to AWS cloud compute.

# AWS Cloud Infrastructure & Production Deployment

## 6.1 AWS Cloud Compute & Network Configuration

HindiMate AI is deployed on Amazon Web Services (AWS) within the **ap-south-1 (Mumbai, India)** region to minimize network round-trip latency for students and exam aspirants across India:

| Infrastructure Component | Configuration Value                                      | Technical & Architectural Justification                                                                                    |
|--------------------------|----------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------|
| Compute Instance         | AWS EC2 <code>t3.micro</code> (1 vCPU, 1.0 GB RAM)       | Burst-capable AWS Nitro System architecture providing cost-effective performance for serving containerized static assets.  |
| Host Operating System    | Amazon Linux 2023 (Kernel 6.1)                           | Lightweight, security-hardened Linux distribution with native Docker and AWS CLI integration.                              |
| Public IPv4 Address      | <code>13.204.64.226</code>                               | Static public network endpoint routed through AWS Internet Gateway.                                                        |
| Storage Volume           | 30 GB General Purpose SSD ( <code>gp3</code> )           | High baseline IOPS (3,000 IOPS / 125 MB/s) handling OS, Docker layers, and container logs.                                 |
| Security Group Ingress   | Inbound: Port 80 (HTTP), Port 22 (SSH), Port 443 (HTTPS) | Restricted ingress firewall rules permitting public web access while safeguarding administrative ports.                    |
| Process Supervision      | Docker Engine 25.x with auto-restart                     | Container configured with <code>--restart unless-stopped</code> policy, ensuring automatic recovery upon instance reboots. |

## 6.2 Production Deployment Verification

The containerized application is actively running on the production host. The deployment was verified through HTTP status inspections, endpoint response latency tests, and container state queries:

```
# AWS EC2 Container Status Verification
$ docker ps --format "table {{.ID}}\t{{.Image}}\t{{.Status}}\t{{.Ports}}"
```

| CONTAINER ID | IMAGE                        | STATUS      | PORTS                           |
|--------------|------------------------------|-------------|---------------------------------|
| 3a9b1c7f4e2d | raghavendra1169/hindimate:01 | Up 18 hours | 0.0.0.0:80→80/tcp, :::80→80/tcp |

```
# HTTP Connectivity & Edge Header Verification
$ curl -I http://13.204.64.226/
HTTP/1.1 200 OK
Server: nginx/1.25.4
Content-Type: text/html; charset=utf-8
Cache-Control: public, max-age=3600
Connection: keep-alive
```

Official Project Deliverables & Production Endpoints:

- Live Production Application: <http://13.204.64.226/>
- Source Code Repository: <https://github.com/Raghavendra1204/linguamate>
- Docker Hub Container Image: [raghavendra1169/hindimate:01](https://hub.docker.com/r/raghavendra1169/hindimate:01)